

Documentación del Modelo de Datos — Mercado Libro

1. Alcance de este documento

Documentación completa del esquema vigente: todas las tablas, atributos, ENUMs, funciones, triggers y vistas — incluyendo las que no cambiaron en esta ronda, porque el pedido es "incluir todo". Además de describir el diseño elegido y su justificación, esta versión agrega una sección dedicada a los **diseños alternativos que se consideraron y se descartaron**, explicando no solo por qué se prefirió el actual sino por qué, dadas las decisiones ya tomadas en el proyecto, los otros diseños **no son opciones correctas** — no es una preferencia de estilo, es una consecuencia de restricciones ya aceptadas (claves naturales, sin identificadores artificiales, integridad referencial declarativa).

Convención de referencias: HU X.Y = historia de usuario del backlog. E1 gestión de libros/intercambios, E2 puntos, E3 usuarios/reputación, E4 búsqueda/seguimiento/notificaciones. RNE = regla de negocio explícita

Documentación del Modelo de Datos — Mercado Libro

2. Convenciones y decisiones transversales	2
3. Tipos de datos personalizados (ENUMs)	3
4. Tablas	5
4.1 usuario	5
4.2 categoria	5
4.3 libro_metadata_cache	6
4.4 publicacion	6
4.5 compra	7
4.6 intercambio	7
4.7 CAMBIO_PUNTOS — entidad conceptual (sin tabla física)	8
4.8 evento_sistema (nueva)	8
4.9 movimiento_puntos_compra (nueva)	9
4.10 movimiento_puntos_intercambio	9
4.11 movimiento_puntos_resena (nueva)	10
4.12 movimiento_puntos_sistema (nueva)	10
4.13 cadena	11
4.14 resena	12
4.15 reporte	13
4.16 baneo / bajar_calificacion / cartel_mal_intercambiador	14
4.16.1 baneo	14
4.16.2 bajar_calificacion	14
4.16.3 cartel_mal_intercambiador	15
4.17 lista	15
4.18 clasificado_en	16
4.19 publicacion_historial_precio	16
4.20 notificacion	17
5. Funciones y triggers	18
6. Flujos de negocio	20
7. Cardinalidades relevantes (MER)	20
8. Limitaciones conocidas y pendientes	21
9. Diseños alternativos considerados y rechazados	21
9.1 Por qué MODIFICA tiene que ser ternaria (no dos relaciones binarias)	21
9.2 Por qué CAMBIO_PUNTOS no necesita un trigger de disjunción	22
9.3 Por qué no un origen polimórfico con clave surrogate (origen_id)	22
9.4 Por qué no tabla física única (tipo_origen) y columnas nullable por tipo	22
9.5 Por qué no una columna origen jsonb	23
9.6 Por qué no guardar los movimientos como columnas fijas en compra/intercambio/resena	23
9.7 Por qué ALTA_INICIAL no es una entidad propia	23
9.8 Por qué RESENA no necesita una entidad envoltorio (RECOMPENSA_RESENA)	24
9.9 Por qué no fusionar COMPRA e INTERCAMBIO en una sola entidad TRANSACCION	24
10. Resumen: fuerte vs. débil, tabla final	25

2. Convenciones y decisiones transversales

Decisión	Justificación
Claves naturales, no surrogates	usuario.email, publicacion(isbn, email_propietario_id, hora_de_publicacion), y todo lo que depende de ellas. No se crean id_compra, id_intercambio, id_movimiento, id_cambio. Única excepciones: notificacion.id (ver §4.15) — no existe otra clave de negocio posible para un evento de notificación individual, y Usuario.id por temas de Auth.
ENUMs para dominios fijos y chicos	Integridad de dominio sin tablas catálogo adicionales. Costo aceptado: agregar un valor requiere ALTER TYPE (o recrear el tipo si hay que quitar un valor, porque Postgres no soporta DROP VALUE).
Especialización disjunta sin tabla padre física	Se usa dos veces en el esquema (sanciones, cambio de puntos), con dos mecanismos distintos según si las claves de los subtipos comparten o no el mismo espacio de identidad (ver §5 y §9).
FK compuestas reales, no columnas sueltas	Cuando el padre tiene PK de varias columnas, la FK se declara como una sola constraint compuesta — nunca varias FKs de una columna cada una, que permitirían combinar valores de filas distintas del padre.
Usuario_id	Es necesario porque Supabase Auth genera en auth.users un uuid inmutable que el esquema no puede reemplazar por email

3. Tipos de datos personalizados (ENUMs)

Tipo	Valores	Para qué
rol_usuario	ADMINISTRADOR, USUARIO	Actor.
estado_cuenta	ACTIVA, SUSPENDIDA, BANEADA	Estado de cuenta; BANEADA es el efecto esperado de un baneo.
calidad_libro	NUEVO, COMO_NUEVO, BUENO, ACEPTABLE, MALO	Estado físico del ejemplar.
estado_intercambio	PENDIENTE, ACEPTADO, RECHAZADO, CANCELADO, COMPLETADO	Ciclo de vida del intercambio directo.
estado_compra	PENDIENTE, ACEPTADA, RECHAZADA, CANCELADA, COMPLETADA	Ciclo de vida de la adquisición con puntos
tipo_movimiento	INGRESO, EGRESO, RESERVA, LIBERACION_RESERVA, DEVOLUCION	Tipo de movimiento contable; es el atributo parcial que completa la PK de cualquier movimiento_puntos_*.
tipo_evento_sistema	ALTA_INICIAL, PROMOCION, AJUSTE_ADMIN, OTRO	Discrimina el tipo de evento dentro de evento_sistema.
calidad_resena	POSITIVA, NEGATIVA	Clasificación adicional de la reseña.
entidad_reporte	USUARIO, PUBLICACION, INTERCAMBIO	Tipo de objeto reportado.
motivo_reporte	MAL_ESTADO_LIBRO, INCUMPLIMIENTO_INTERCAMBIO, FRAUDE, COMPORTAMIENTO_INADECUADO, OTRO	Motivo del reporte.
estado_reporte	PENDIENTE, EN_REVISION, RESUELTO, RECHAZADO	Flujo de moderación.

Tipo	Valores	Para qué
estado_cadena	ACTIVA, COMPLETADA, CANCELADA	Ciclo de vida de una cadena de intercambios.
estado_publicacion	DISPONIBLE, RESERVADA, VENDIDA, ELIMINADA	Reemplaza al booleano disponibilidad.
color_semaforo	VERDE, AMARILLO, ROJO, SIN_REFERENCIA	Resultado del semáforo de precio.
frecuencia_notificacion	INSTANTANEA, DIARIA	HU 4.6.1.
tipo_notificacion	PUBLICACION_NUEVA, BAJA_PRECIO, CRUCE_VERDE, OTRO	Origen del evento de notificación.

Eliminado: tipo_intercambio (DIRECTO, CON_PUNTOS). Ya no tiene función: INTERCAMBIO ahora representa exclusivamente el intercambio directo; la modalidad de puntos vive enteramente en COMPRA. Mantenerlo habría dejado una columna que siempre iba a valer lo mismo, y una ambigüedad real (¿qué tabla gobierna una transacción con puntos, intercambio.tipo='CON_PUNTOS' o compra?) que ahora no existe.

4. Tablas

4.1 usuario

Columna	Tipo	Notas
email	varchar	
usuario_id	uuid	pk
nombre_usuario	varchar	
rol	rol_usuario	
saldo_total	numeric	CHECK >= 0
saldo_reservado	numeric	CHECK >= 0 y <= saldo_total
reputacion_promedio	numeric	nullable, CHECK 1-5
estado_cuenta	estado_cuenta	default ACTIVA
notificacion_email / notificacion_inapp	bool	default true
frecuencia_notificacion	frecuencia_notificacion	default INSTANTANEA
hora_resumen_diario	time	nullable

4.2 categoria

Columna	Tipo	Notas
nombre	varchar	PK
categoria_padre_id	varchar	FK a categoria(nombre), nullable

Auto-referenciada; NULL = raíz. CHECK evita que una categoría sea padre de sí misma (no bloquea ciclos indirectos).

4.3 libro_metadata_cache

Columna	Tipo	Notas
isbn	varchar	PK
google_books_id	varchar	UNIQUE NOT NULL
titulo	varchar	
autores	varchar	nullable
puntuacion_externa	numeric	nullable, 0–5
fecha_cache_bibliografico / fecha_cache_puntuacion	timestamp	nullable, control de frescura independiente

Cache de una fuente externa (Google Books); evita llamadas repetidas.

4.4 publicacion

Columna	Tipo	Notas
isbn, email_propietario_id, hora_de_publicacion	—	PK, FK a libro_metadata_cache y usuario
estado_fisico	calidad_libro	
valor_puntos_solicitado	bigint	CHECK >= 0
valor_referencia_calculado	bigint	nullable, CHECK >= 0
comentario	varchar	nullable
estado	estado_publicacion	default DISPONIBLE
color_semaforo	color_semaforo	default SIN_REFERENCIA

Estado reemplaza al booleano original para distinguir vendida/eliminada/reservada (HU 4.6.3); el borrado lógico es lo que permite que notificacion y publicacion_historial_precio sigan apuntando a la fila con integridad referencial normal.

4.5 compra

Columna	Tipo	Notas
comprador_id, isbn, propietario_id, hora_de_publicacion	—	PK; FK compuesta a usuario y a publicacion
puntos	bigint	CHECK > 0
estado	estado_compra	default PENDIENTE — nuevo

Se agrega estado. Antes no existía ningún lugar para registrar que una solicitud de compra fue aceptada, rechazada o cancelada (HU 1.7 no tenía dónde vivir para la modalidad de puntos). Ahora sigue el mismo patrón que intercambio.estado.

4.6 intercambio

Columna	Tipo	Notas
isbn_solicitante / propietario_id_solicitante / hora_de_publicacion_solicitante	—	PK (parte 1), FK a publicacion
isbn_ofrecida / propietario_id_ofrecida / hora_de_publicacion_ofrecida	—	PK (parte 2), FK a publicacion
estado	estado_intercambio	
puntos_comprometidos	numeric	default 0, CHECK >= 0 (compensación en puntos dentro de un trueque desigual)
tipo	tipo_intercambio	Eliminado — ver §3

Entidad débil identificada por la relación identificadora doble "Acepta" (dos ocurrencias de PUBLICACION). CHECK de tuplas distintas (publicación no puede ofertarse contra sí misma) y CHECK propietario_id_solicitante <> propietario_id_ofrecida (nadie intercambia consigo mismo con dos libros propios).

4.7 CAMBIO_PUNTOS — entidad conceptual (sin tabla física)

No existe como tabla. Es la generalización lógica de cuatro orígenes disjuntos de un movimiento de puntos:

CAMBIO_PUNTOS (concepto, sin PK propia)

├— COMPRA (entidad fuerte)

├— INTERCAMBIO (entidad débil)

├— RESEÑA (entidad débil)

└— SISTEMA (entidad fuerte)

La generalización es **unión** (un movimiento tiene exactamente un origen) y **total por construcción**: como CAMBIO_PUNTOS no tiene tabla propia, no existe forma de crear un "origen" que no sea, al mismo tiempo, una fila real de una de las cuatro tablas. La totalidad no necesita enforcement — es una consecuencia estructural de no tener supertipo físico.

4.8 evento_sistema (nueva)

Columna	Tipo	Notas
tipo_evento	tipo_evento_sistema	PK (parte 1)
fecha_evento	timestamp	PK (parte 2), default now()
descripcion	varchar	nullable

Catch-all para orígenes sin clave natural propia: alta inicial (automática, ver §6), promociones masivas, ajustes administrativos. **Riesgo documentado**: dos eventos del mismo tipo_evento en el mismo instante colisionarían contra la PK.

4.9 movimiento_puntos_compra (nueva)

Columna	Tipo	Notas
comprador_id, isbn, propietario_id, hora_de_publicacion	—	PK (parte 1), FK compuesta a compra
id_usuario	uuid	PK (parte 2), FK a usuario
tipo	tipo_movimiento	PK (parte 3)
monto	bigint	CHECK > 0

id_usuario debe ser comprador_id o propietario_id de esa compra —
trg_validar_usuario_movimiento_compra.

4.10 movimiento_puntos_intercambio

Columna	Tipo	Notas
isbn_solicitante	varchar	PK
propietario_id_solicitante	varchar	PK
hora_de_publicacion_solicitante	timestamp	PK
isbn_ofrecida	varchar	PK
propietario_id_ofrecida	varchar	PK
hora_de_publicacion_ofrecida	timestamp	PK
id_usuario	uuid	PK
tipo	tipo_movimiento	PK
monto	int8	

4.11 movimiento_puntos_resena (nueva)

Columna	Tipo	Notas
(7 columnas de resena, incluido email_reviewer)	—	PK (parte 1), FK compuesta a resena
id_usuario	uuid	PK (parte 2), FK a usuario
tipo	tipo_movimiento	PK (parte 3)
monto	bigint	CHECK > 0
Solicitante_reviwer	Booleano	

Referencia a RESENA **directamente** — no hay una entidad "RECOMPENSA_RESENA" intermedia (ver §9 para la justificación).

4.12 movimiento_puntos_sistema (nueva)

Columna	Tipo	Notas
tipo_evento, fecha_evento	—	PK (parte 1), FK compuesta a evento_sistema
id_usuario	uuid	PK (parte 2), FK a usuario
tipo	tipo_movimiento	PK (parte 3)
monto	bigint	CHECK > 0

Sin trigger de participante: el destinatario de un evento de sistema lo decide la aplicación (a quién se le acredita una promoción, por ejemplo) — no hay una regla estructural de "quién puede" como sí la hay en compra/intercambio/reseña.

4.13 cadena

Columna	Tipo	Notas
isbn_anterior	varchar	PK, parte de FK compuesta a intercambio
propietario_id_anterior	varchar	PK, parte de FK compuesta a intercambio
hora_de_publicacion_anterior	timestamp	PK, parte de FK compuesta a intercambio
isbn_siguiente	varchar	PK, parte de FK compuesta a intercambio
propietario_id_siguiente	varchar	PK, parte de FK compuesta a intercambio
hora_de_publicacion_siguiente	timestamp	PK, parte de FK compuesta a intercambio
estado	estado_cadena	Default ACTIVA

Las dos referencias a intercambio se implementan como **FKs compuestas completas**, una para el intercambio anterior y otra para el intercambio siguiente. No se utilizan FKs independientes por columna.

La continuidad de la cadena se garantiza mediante un CHECK que exige que la publicación ofrecida en el intercambio anterior coincida con la publicación solicitada en el intercambio siguiente:

ofrecida_anterior = solicitante_siguiente.

De esta forma, cadena no representa simplemente dos intercambios relacionados, sino que garantiza que el libro entregado en un eslabón sea el libro que recibe como solicitado el siguiente participante.

El atributo estado representa el ciclo de vida de la cadena mediante los valores ACTIVA, COMPLETADA y CANCELADA.

4.14 resena

Columna	Tipo	Notas
isbn_solicitante	varchar	PK, parte de FK compuesta a intercambio
propietario_id_solicitante	varchar	PK, parte de FK compuesta a intercambio
hora_de_publicacion_solicitante	timestamp	PK, parte de FK compuesta a intercambio
isbn_ofrecida	varchar	PK, parte de FK compuesta a intercambio
propietario_id_ofrecida	varchar	PK, parte de FK compuesta a intercambio
hora_de_publicacion_ofrecida	timestamp	PK, parte de FK compuesta a intercambio
Solicitante_reviewer	boolean	PK, booleano
calificacion	int2	CHECK entre 1 y 5
comentario	varchar	Nullable
calidad	calidad_resena	Nullable

No existe una columna que almacene explícitamente al usuario reseñado. El usuario reseñado se determina por exclusión: dado que un intercambio tiene dos participantes, es el participante que no coincide. Esta lógica se obtiene mediante el booleano solicitante_reviewer, if True el reseñante es el mismo que el solicitante if False entonces el reviewer es el ofreciente.

La inserción y actualización de una reseña están protegidas por trg_validar_resena, que exige que el intercambio se encuentre en estado COMPLETADO.

Después de insertar, actualizar o eliminar una reseña, trg_reputacion_resena recalcula usuario.reputacion_promedio para el usuario afectado.

Además, resena constituye uno de los cuatro posibles tipos de origen conceptual de CAMBIO_PUNTOS. Cuando una reseña genera una recompensa, el movimiento correspondiente se registra en movimiento_puntos_resena, sin crear una entidad intermedia adicional.

4.15 reporte

Columna	Tipo	Notas
email_reportante_id	varchar	PK, FK a usuario
hora_reporte	timestamp	PK
email_reportado_id	varchar	PK, FK a usuario
entidad_tipo	entidad_reporte	Tipo de entidad reportada
motivo	motivo_reporte	Motivo del reporte
estado	estado_reporte	Default PENDIENTE

email_reportante_id referencia al usuario que realiza el reporte y email_reportado_id al usuario reportado.

La tabla permite indicar mediante entidad_tipo si el objeto del reporte corresponde a un USUARIO, PUBLICACION o INTERCAMBIO.

El motivo se restringe mediante el ENUM motivo_reporte, con los valores MAL_ESTADO_LIBRO, INCUMPLIMIENTO_INTERCAMBIO, FRAUDE, COMPORTAMIENTO_INADECUADO y OTRO.

El ciclo de moderación se almacena en estado, cuyos valores son PENDIENTE, EN_REVISION, RESUELTO y RECHAZADO.

Se mantiene un CHECK que impide que email_reportante_id y email_reportado_id sean iguales.

4.16 baneo / bajar_calificacion / cartel_mal_intercambiador

Las tres tablas representan los posibles subtipos de sanción originados por un reporte. Son entidades débiles dependientes directamente de reporte.

4.16.1 baneo

Columna	Tipo	Notas
email_reportante_id	varchar	PK, parte de FK compuesta a reporte
hora_reporte	timestamp	PK, parte de FK compuesta a reporte
email_reportado_id	varchar	PK, parte de FK compuesta a reporte
fecha_inicio	timestamp	NOT NULL, default now()
fecha_fin	timestamp	Nullable

La relación con reporte se implementa mediante una FK compuesta completa.

fecha_fin IS NULL representa un baneo indefinido o actualmente vigente.

4.16.2 bajar_calificacion

Columna	Tipo	Notas
email_reportante_id	varchar	PK, parte de FK compuesta a reporte
hora_reporte	timestamp	PK, parte de FK compuesta a reporte
email_reportado_id	varchar	PK, parte de FK compuesta a reporte
castigo	int2	Nullable; magnitud de la penalización

La PK coincide con la PK de reporte y existe una FK compuesta completa hacia dicha tabla.

castigo representa la magnitud de la penalización aplicada a la reputación del usuario reportado.

4.16.3 cartel_mal_intercambiador

Columna	Tipo	Notas
email_reportante_id	varchar	PK, parte de FK compuesta a reporte
hora_reporte	timestamp	PK, parte de FK compuesta a reporte
email_reportado_id	varchar	PK, parte de FK compuesta a reporte
cartel	bool	Default true

Cartel indica que el usuario debe aparecer marcado como mal intercambiador en su perfil.

Las tres tablas forman una especialización **disjunta y parcial** de reporte: un reporte puede no producir ninguna sanción o producir exactamente un tipo de sanción. La disjunción se garantiza mediante `fn_validar_sancion_disjunta` y sus triggers asociados, que impiden que la misma clave de reporte aparezca simultáneamente en dos de las tres tablas.

4.17 lista

Columna	Tipo	Notas
email_usuario	varchar	PK, FK a usuario
isbn	varchar	PK, FK a libro_metadata_cache
nota_privada	varchar	Nullable
precio_min	int	Nullable
precio_max	int	Nullable
condiciones_aceptables	calidad_libro[]	Nullable
fecha_agregado	timestamp	Default now()

La tabla representa el seguimiento de un libro por parte de un usuario.

`nota_privada` permite almacenar una anotación del usuario. `precio_min` y `precio_max` representan el rango de puntos que el usuario está dispuesto a aceptar. Cuando ambos están definidos, se exige que `precio_min` $\leq$ `precio_max`.

condiciones_aceptables es un array del ENUM calidad_libro, por lo que puede almacenar varias condiciones aceptables sin necesidad de una tabla auxiliar. Por ejemplo, un usuario puede aceptar {NUEVO, COMO_NUEVO, BUENO}.

4.18 clasificado_en

Columna	Tipo	Notas
isbn	varchar	PK, FK a libro_metadata_cache
nombre_categoria	varchar	PK, FK a categoria(nombre)

usuario_id implementa la relación N:M entre libro_metadata_cache y categoria. isbn referencia al libro bibliográfico y nombre_categoria referencia a una categoría existente. La clasificación pertenece al libro identificado por ISBN y no a una publicación concreta. Por ello, todas las publicaciones de un mismo ISBN comparten la clasificación bibliográfica. La referencia a categoria puede apuntar tanto a una categoría raíz como a una subcategoría, ya que la jerarquía se representa mediante la auto-referencia categoria.categoria_padre_id.

4.19 publicacion_historial_precio

Columna	Tipo	Notas
isbn	varchar	PK, parte de FK compuesta a publicacion
email_propietario_id	varchar	PK, parte de FK compuesta a publicacion
hora_de_publicacion	timestamp	PK, parte de FK compuesta a publicacion
fecha_cambio	timestamp	PK, default now()
valor_puntos_anterior	int	Valor anterior
valor_puntos_nuevo	int	Valor nuevo
color_anterior	color_semaforo	Nullable
color_nuevo	color_semaforo	Nullable

La tabla funciona como una bitácora: cada modificación de precio genera un registro independiente. fecha_cambio forma parte de la PK porque una misma publicación puede modificar su precio varias veces.

color_anterior y color_nuevo permiten identificar cambios del semáforo, incluyendo el cruce desde un color distinto de VERDE hacia VERDE. Los registros se generan automáticamente mediante trg_trackear_cambio_precio; la aplicación no debe insertar directamente las filas de esta tabla.

La tabla conserva el historial necesario para las reglas de detección de baja de precio y cruce a verde utilizadas por las notificaciones.

4.20 notificacion

Columna	Tipo	Notas
id	bigint	PK, GENERATED ALWAYS AS IDENTITY
email_usuario	varchar	FK a usuario
isbn	varchar	Parte de FK compuesta a publicacion
email_propietario_id	varchar	Parte de FK compuesta a publicacion
hora_de_publicacion	timestamp	Parte de FK compuesta a publicacion
tipo	tipo_notificacion	Tipo de evento que originó la notificación
leida	bool	Default false
archivada	bool	Default false
fecha_creacion	timestamp	Default now()

id es la única clave primaria artificial del esquema. Se utiliza GENERATED ALWAYS AS IDENTITY. Esta referencia se realiza contra publicacion y no contra libro_metadata_cache, porque la notificación debe conservar el vínculo con el ejemplar concreto que originó el evento. El vínculo puede mantenerse incluso cuando la publicación posteriormente se vende o elimina porque publicacion utiliza borrado lógico mediante estado = ELIMINADA en lugar de eliminar físicamente la fila. tipo identifica el evento que originó la notificación. Sus valores son PUBLICACION_NUEVA, BAJA_PRECIO, CRUCE_VERDE y OTRO. leida indica si el usuario ya leyó la notificación y archivada permite retirarla de la vista activa sin eliminar el registro. Las preferencias sobre el canal y frecuencia de envío no pertenecen a esta tabla: notificacion_email, notificacion_inapp, frecuencia_notificacion y hora_resumen_diario se mantienen en usuario, porque representan configuración del usuario y no características del evento de notificación. La aplicación crea las notificaciones a partir de los eventos

detectados en publicaciones nuevas, cambios de precio y cruces a verde. La tabla conserva el evento para que pueda seguir consultándose aunque el estado de la publicación cambie posteriormente.

5. Funciones y triggers

Función / Trigger	Tabla	Dispara en	Qué garantiza
fn_validar_usuario_movimiento_compra / trg_validar_usuario_movimiento_compra	movimiento_puntos_compra	BEFORE INSERT/UPDATE	$id_usuario \in \{comprador_id, propietario_id\}$
fn_validar_usuario_movimiento_intercambio / trg_validar_usuario_movimiento_intercambio	movimiento_puntos_intercambio	BEFORE INSERT/UPDATE	$id_usuario \in \{propietario_id_solicitante, propietario_id_ofrecida\}$
— (ninguno)	movimiento_puntos_sistema	—	Destinatario libre, definido por la aplicación
fn_puntos_alta_inicial	— (helper)	—	Devuelve el monto de alta inicial (centraliza la regla de negocio)
fn_alta_inicial_automatica / trg_alta_inicial_automatica	usuario	AFTER INSERT	Crea el evento en evento_sistema, el movimiento en movimiento_puntos_sistema, y acredita saldo_total — todo automático, sin tabla propia para "alta inicial"
fn_validar_sancion_disjunta / trg_disjunta_baneo, trg_disjunta_bajar_calificacion, trg_disjunta_cartel	baneo, bajar_calificacion, cartel_mal_intercambiador	BEFORE INSERT	Un mismo reporte no puede estar en más de un subtipo
fn_validar_resena / trg_validar_resena	resena	BEFORE INSERT/UPDATE	Intercambio COMPLETADO
fn_resena_reviewed	resena	— (helper)	Determina el usuario reseñado a partir de solicitante_reviewer: si es

Función / Trigger	Tabla	Dispara en	Qué garantiza
			TRUE, el reviewer es el solicitante; si es FALSE, el reviewer es el propietario de la publicación ofrecida.
fn_recalcular_reputacion / fn_actualizar_reputacion / trg_reputacion_resena	resena / usuario	AFTER INSERT/UPDATE /DELETE	Recalcula usuario.reputacion_promedio del usuario reseñado, determinado mediante fn_resena_reviewed.
fn_calcular_color_semaforo, fn_set_color_inicial / trg_set_color_inicial	publicacion	BEFORE INSERT	Calcula el color inicial del semáforo
fn_trackear_cambioPrecio / trg_trackear_cambioPrecio	publicacion	BEFORE UPDATE	Recalcula color y registra el cambio en publicacion_historial_categoria
fn_evitar_ciclo_categoria/trg_evitar_ciclo_categoria	Categoría	BEFORE INSERT/UPDATE	Verifica que no hayan ciclos autorreferentes

6. Flujos de negocio

Alta de un usuario (HU 2.1): INSERT en usuario → trg_alta_inicial_automatica corre sola: crea el evento en evento_sistema, el movimiento en movimiento_puntos_sistema, y suma el monto a saldo_total. Cero pasos manuales.

Adquirir con puntos (HU 1.5, 2.4): INSERT en compra (estado = PENDIENTE) → aceptar/rechazar actualiza estado → si se acepta, INSERT en movimiento_puntos_compra (RESERVA para el comprador) → al completarse, INSERT adicional (EGRESO comprador, INGRESO propietario) o, si se cancela, LIBERACION_RESERVA.

Intercambio directo (HU 1.6, 1.8, 1.10): sin cambios respecto a la documentación previa, ahora contra movimiento_puntos_intercambio.

Puntos por reseña (HU 2.3, parcial): INSERT en resena (ya validado por trg_validar_resena) → si corresponde recompensa, INSERT en movimiento_puntos_resena — la propia PK impide que la misma reseña genere el crédito dos veces.

Ajustes y promociones: INSERT manual en evento_sistema + movimiento_puntos_sistema, mismo mecanismo que alta inicial pero disparado por un administrador en vez de un trigger.

7. Cardinalidades relevantes (MER)

Relación	Cardinalidad	Nota
MODIFICA: USUARIO – CAMBIO_PUNTOS – MOVIMIENTO_PUNTOS	1 : 1 : n	Ternaria, identificadora. Ver §4.14 y §9.1
CAMBIO_PUNTOS ⊃ {COMPRA, INTERCAMBIO, RESENA, SISTEMA}	especialización disjunta, total por construcción	Ver §4.7
REPORTE ⊃ {BANEADO, BAJAR_CALIFICACION, CARTEL_MAL_INTERCAMBIADOR}	especialización disjunta, parcial, con trigger	Ver §9.2

8. Limitaciones conocidas y pendientes

- Las funciones que actualizan saldo_total/saldo_reservado a partir de movimiento_puntos_compra/_intercambio/_reseña no están automatizadas — solo el trigger de alta inicial actualiza el saldo por sí solo. Para compra/intercambio/reseña, la actualización de saldo sigue siendo responsabilidad de la aplicación al insertar el movimiento.
- El umbral del semáforo ($\pm 10\%$) es un placeholder pendiente de definición de negocio.
- La inserción en baneo no actualiza automáticamente usuario.estado_cuenta.
- La vista movimiento_puntos es de solo lectura.

9. Diseños alternativos considerados y rechazados

Esta sección no es un listado de gustos — cada alternativa se rechaza por una razón concreta derivada de restricciones que el proyecto ya había fijado (claves naturales, sin surrogates, integridad referencial declarativa) o de reglas del modelo entidad-relación que no admiten alternativa una vez que se conocen los hechos del dominio.

9.1 Por qué MODIFICA tiene que ser ternaria (no dos relaciones binarias)

Alternativa rechazada: modelar MOVIMIENTO_PUNTOS–CAMBIO_PUNTOS como una relación identificadora binaria, y por separado, MOVIMIENTO_PUNTOS–USUARIO como una relación binaria común (no identificadora), con id_usuario como un atributo cualquiera de MOVIMIENTO_PUNTOS.

Por qué no funciona: una entidad débil, por definición del modelo entidad-relación, tiene **una sola** relación identificadora — es la que le presta la parte de la clave que ella no puede generar por sí misma. La clave de MOVIMIENTO_PUNTOS necesita **dos** préstamos distintos: la identidad del origen (de CAMBIO_PUNTOS) y a quién se le aplica (de USUARIO). Si esos dos préstamos vinieran de dos relaciones distintas, MOVIMIENTO_PUNTOS tendría dos relaciones identificadoras — que no es una construcción válida en el modelo. La única forma correcta de expresar "mi clave depende de dos dueños distintos a la vez" es una relación identificadora que toque a los dos dueños simultáneamente: eso es, por definición, ternaria (tres puntas: los dos dueños + la entidad débil).

Prueba de que el segundo préstamo (usuario) es necesario y no un atributo cualquiera: si id_usuario no formara parte de la relación identificadora (y por lo tanto no formara parte de la PK), un mismo origen (por ejemplo, un intercambio) no podría tener movimientos de tipo RESERVA para **ambos** propietarios al mismo tiempo — la PK (origen, tipo) sola colisionaría en la segunda inserción. Como el negocio sí necesita eso (un intercambio con puntos_comprometidos puede generar una RESERVA para cada uno de los dos participantes), id_usuario tiene que ser parte de la clave, y por lo tanto tiene que venir de la misma relación identificadora que CAMBIO_PUNTOS. No hay una forma alternativa de lograr esto sin volver ternaria la relación.

9.2 Por qué CAMBIO_PUNTOS no necesita un trigger de disjunción

Podría parecer inconsistente que baneo/bajar_calificacion/cartel_mal_intercambiador necesiten un trigger explícito para garantizar disjunción, y COMPRA/INTERCAMBIO/RESENA/SISTEMA no. No es una inconsistencia: es una diferencia real entre los dos casos.

Las tres tablas de sanción comparten **exactamente la misma clave** — la PK de reporte. Como las tres tablas "compiten" por el mismo espacio de identidad, nada impide, a nivel de estructura, insertar la misma clave en dos de las tres — de ahí que haga falta un trigger que lo revise explícitamente.

Las cuatro ramas de CAMBIO_PUNTOS **no comparten espacio de identidad**: la clave de movimiento_puntos_compra viene de compra, la de movimiento_puntos_intercambio viene de intercambio, la de movimiento_puntos_resena viene de resena, la de movimiento_puntos_sistema viene de evento_sistema — son dominios de clave estructuralmente distintos (columnas distintas, tipos de origen distintos). No existe ninguna fila que pueda "aparecer" simultáneamente en dos de estas cuatro tablas porque ni siquiera tienen la misma forma de clave. La disjunción, acá, es gratis — no porque se decidiera no chequearla, sino porque no hay nada que revisar.

9.3 Por qué no un origen polimórfico con clave surrogate (origen_id)

Alternativa considerada en una ronda anterior de este mismo diseño: una tabla origen_movimiento con PK artificial origen_id, y tablas de enlace (origen_compra, origen_intercambio, etc.) que conectan ese surrogate con la clave real de cada tipo.

Por qué se rechaza: resuelve el problema técnico, pero introduce exactamente el tipo de identificador artificial que el proyecto excluyó desde el principio del script original ("no se crea id_compra", "no existe id_intercambio"). Aceptar un surrogate acá, después de haber evitado sistemáticamente cualquier surrogate en el resto del esquema (incluso donde habría sido más simple, como en publicacion o intercambio), sería inconsistente: la regla "no usamos identificadores artificiales" dejaría de ser una regla del proyecto y pasaría a ser "no los usamos, excepto cuando es más cómodo" — lo cual vacía de sentido haberla adoptado en primer lugar.

9.4 Por qué no tabla física única (tipo_origen) y columnas nullable por tipo

Alternativa: una sola tabla movimiento_puntos con columna tipo_origen (ENUM) y todas las columnas de clave posibles (las de compra, las de intercambio, las de reseña, las de sistema) como nullable, usando solo el subconjunto que corresponda según tipo_origen.

Por qué se rechaza, con razones técnicas concretas:

1. **Ninguna FK puede aplicarse correctamente.** Postgres no soporta FKs condicionales ("esta columna solo debe respetar esta FK si tipo_origen = 'COMPRA'"). La única forma de mantener la integridad sería a través de triggers que reimplementen a mano lo que una FK ya hace gratis — es decir, se pierde exactamente la ventaja de tener FKs declarativas, que es lo que sostiene la integridad de todo el resto del esquema.

2. **Viola la tercera forma normal.** Cualquier fila tendría columnas que no dependen de su propia clave completa, sino del valor de `tipo_origen` — es una dependencia parcial encubierta, el mismo problema que la normalización busca evitar.
3. **No escala.** Agregar un quinto origen en el futuro implicaría un `ALTER TABLE` sobre una tabla ya en producción, en vez de un simple `CREATE TABLE` nuevo — más riesgo, más bloqueo de tabla, migración de datos innecesaria.

9.5 Por qué no una columna origen jsonb

Alternativa: guardar la clave del origen como JSON semi-estructurado en una sola columna.

Por qué se rechaza: Postgres no permite declarar una `FOREIGN KEY` sobre el contenido de una columna `jsonb`. Toda la integridad referencial pasaría a ser responsabilidad exclusiva de la aplicación, sin ninguna garantía a nivel de base de datos de que el origen referenciado realmente existe. Es la misma objeción que en 9.4 pero más severa, porque ni siquiera hay tipado de columnas — cualquier estructura mal formada pasa silenciosamente.

9.6 Por qué no guardar los movimientos como columnas fijas en compra/intercambio/resena

Alternativa: en vez de una fila por movimiento, columnas como `compra.monto_reservado`, `compra.monto_liberado`, `compra.monto_transferido`.

Por qué se rechaza: el número de movimientos que puede generar un origen **no está acotado de antemano** — una reserva puede liberarse y volver a reservarse, o generar una devolución parcial más adelante; HU 2.6 exige poder auditar cada movimiento individual con su fecha y tipo. Un conjunto fijo de columnas solo puede representar una cantidad fija y conocida de eventos; acá la cantidad es variable, así que hace falta una tabla con una fila por evento.

9.7 Por qué `ALTA_INICIAL` no es una entidad propia

Diseño previo, ya descartado: una entidad débil `ALTA_INICIAL`, identificada 1:1 por `USUARIO`, con su propia tabla `movimiento_puntos_alta`.

Por qué se descarta: el alta inicial no tiene ninguna variabilidad de negocio — ocurre exactamente una vez, para todo usuario, sin excepciones ni decisiones intermedias (no hay un "aceptar/rechazar" como en compra o intercambio). Crear una entidad y una tabla completas para un evento que es enteramente automático y 1:1 con `USUARIO` es más estructura de la que el caso necesita. `SISTEMA` ya existe como origen genérico para hechos sin clave natural propia — reutilizarlo no pierde trazabilidad (sigue quedando un `evento_sistema` + un `movimiento_puntos_sistema` por cada alta) y evita una tabla que solo repetiría la clave de usuario sin agregar nada propio.

9.8 Por qué RESENA no necesita una entidad envoltorio (RECOMPENSA_RESENA)

Diseño previo, también descartado: una entidad débil RECOMPENSA_RESENA, identificada 1:1 por RESENA, intercalada entre RESENA y movimiento_puntos_resena.

Por qué se descarta: a diferencia de COMPRA e INTERCAMBIO (que sí pueden generar varios movimientos en el tiempo — reserva, luego liberación o transferencia), una reseña genera el crédito **una sola vez**. Una entidad envoltorio solo tendría sentido si hiciera falta representar "esta reseña, como hecho generador de puntos, puede tener múltiples eventos asociados" — pero ese no es el caso acá. La propia PK de movimiento_puntos_resena (que ya incluye la clave completa de resena) es suficiente para garantizar que el crédito ocurra una sola vez; agregar una entidad intermedia sería duplicar la misma clave sin ganar nada.

9.9 Por qué no fusionar COMPRA e INTERCAMBIO en una sola entidad TRANSACCION

Alternativa considerada: dado que ambos representan "una modalidad de intercambio", modelarlos como subtipos de una entidad TRANSACCION común, con intercambio conservando el rol de tipo "directo" y compra el de tipo "con puntos".

Por qué se rechaza: INTERCAMBIO y CADENA, RESENA ya están construidas sobre la clave natural específica de intercambio (6 columnas: dos publicaciones completas). Introducir una entidad TRANSACCION intermedia obligaría a decidir de qué se compone SU clave — y como COMPRA (4 columnas) e INTERCAMBIO (6 columnas) no comparten forma, la única manera de darle una clave común a TRANSACCION sería, otra vez, un surrogate — llevando de nuevo al problema de 9.3, ahora arrastrando también a CADENA y RESENA, que tendrían que migrar su clave. El costo de este cambio es mucho mayor que el problema que resuelve (que ya está resuelto de otra forma: CAMBIO_PUNTOS los generaliza igual, sin tocar sus claves).

10. Resumen: fuerte vs. débil, tabla final

Entidad	Fuerte / Débil	Identificada por
USUARIO, CATEGORIA, LIBRO_METADATA_CACHE, REPORTE, EVENTO_SISTEMA, COMPRA	Fuerte	Clave natural propia
PUBLICACION, INTERCAMBIO, CADENA, RESENA, BANEIO/BAJAR_CALIFICACION/ CARTEL_MAL_INTERCAMBIADOR	Débil	Su dueño respectivo
MOVIMIENTO_PUNTOS (4 tablas físicas)	Débil	Relación ternaria identificadora MODIFICA (USUARIO + CAMBIO_PUNTOS)
NOTIFICACION	Fuerte, con PK artificial (única excepción)	—
CAMBIO_PUNTOS	Conceptual — no es ni fuerte ni débil, no tiene tabla	Se manifiesta a través de sus 4 ramas